

Building Accessible Android Applications

A Case Study on an Accessible Podcast & Audiobook App
built with Jetpack Compose and Kotlin



*A technical report on semantic accessibility, screen-reader support,
and accessibility testing in modern Android UI development.*

Group of 4

Member	Student ID
Nguyễn Chí Tính	24125019
Khương	24125062
Phong	24125072
Phát	24125073

Contents

1	Introduction	2
1.1	Why accessibility	2
1.2	Goal and scope	2
1.3	Acceptance bar	2
2	Background	2
2.1	The legal driver	2
2.2	The POUR principles	2
2.3	Two trees, one screen	3
2.4	Jetpack Compose	4
3	The Case-Study App: <i>Accessible Podcast</i>	4
3.1	Overview	4
3.2	User flow	4
3.3	Architecture	4
3.4	The mock data layer	5
3.5	Where each technique lives	5
4	Technical Approach I — Core Semantics	6
4.1	Content descriptions: naming visual elements	6
4.2	Roles and state descriptions on custom controls	6
4.3	Touch-target sizing	7
4.4	Scalable typography	7
4.5	Heading navigation	7
5	Technical Approach II — Advanced Semantics & Navigation	7
5.1	Semantic merging: fewer, richer focus stops	8
5.2	Clearing and hiding: <code>clearAndSetSemantics</code> & <code>hideFromAccessibility</code>	9
5.3	Custom accessibility actions	9
5.4	Traversal order	9
5.5	Decision matrix	10
6	Testing & Evaluation	10
6.1	Manual testing with TalkBack	11
6.2	Accessibility Scanner — and an honest case study	11
6.3	Layout Inspector, Compose UI Testing, and ATF	11
6.4	Evaluation metrics and trade-offs	12

7 Conclusion	13
Appendix on AI usage	13
8 Appendix A — Running the app	13
Appendix A — Running the app	13
Limitations	14
References	15

1 Introduction

1.1 Why accessibility

About one in six people lives with a disability that changes how they use a phone; for a blind user the screen reader *is* the interface. Accessibility is the practice of building software so these users get equal access — and since June 2022 (EAA, §2.1) it is a legal requirement in the EU, not a courtesy.

We chose a podcast/audiobook player on purpose: audio is one of the few digital experiences a blind user can consume on exactly equal terms with a sighted user — *if* the surrounding controls are reachable. The entire accessibility gap here lives in the UI, which makes it a clean case study.

1.2 Goal and scope

We took a conventional, accessibility-blind podcast UI and rebuilt it to be fully operable by TalkBack, Switch Access, and 200% fonts, justifying every technique. The deliverable is a front-end-only Jetpack Compose app (data mocked behind a repository interface — no backend) plus this report, in four parts:

- **Background** (§2) — the standards and the Android accessibility model.
- **Core semantics** (§4) — labels, roles, state, touch targets, fonts.
- **Advanced semantics** (§5) — merging, traversal order, custom actions.
- **Testing & evaluation** (§6) — verification and honest trade-offs.

1.3 Acceptance bar

One concrete target drove every decision: *a user who never looks at the screen can open the app, browse shows, pick an episode, and control playback with TalkBack alone*. Figure 1 is what a sighted user sees; the rest of the report is what a screen-reader user hears at each step.

2 Background

2.1 The legal driver

The **European Accessibility Act** (Directive (EU) 2019/882) requires products sold in the EU to meet common accessibility requirements; every member state transposed it into national law by June 2022. Its technical yardstick is **WCAG 2.2**. WCAG targets the web, so for a native app we apply it through *WCAG2ICT* and mobile guidance — mapping each success criterion to its Android equivalent rather than claiming literal conformance.

2.2 The POUR principles

WCAG rests on four principles (Figure 2). For a media player they read concretely as:

- **Perceivable** — label every transport icon; keep duration text at $\geq 4.5:1$ contrast.
- **Operable** — play, seek, and bookmark reachable from TalkBack, not only by touch.
- **Understandable** — the play button announces “playing” vs. “paused”.
- **Robust** — expose a correct semantics tree instead of relying on pixels.

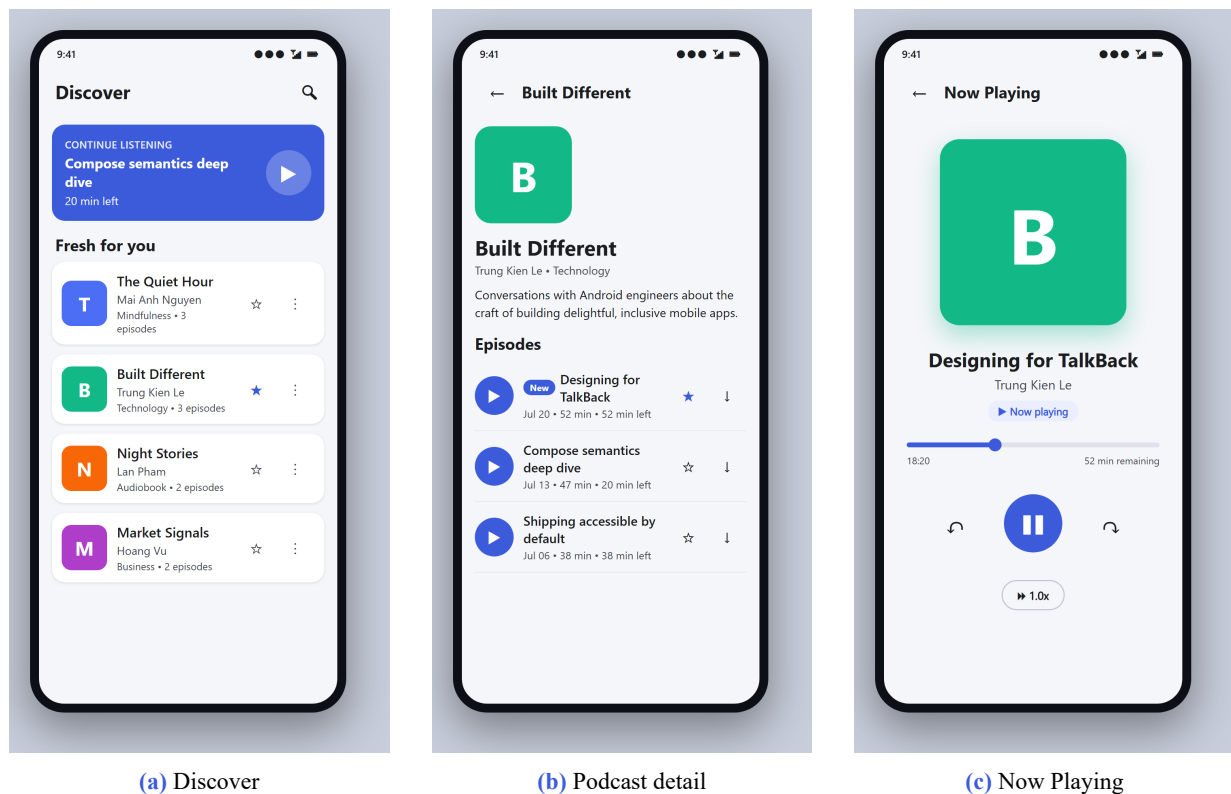


Figure 1: The three screens of the case-study app: **Discover** → **Podcast detail** → **Now Playing**, plus a “Continue listening” shortcut on the home screen.

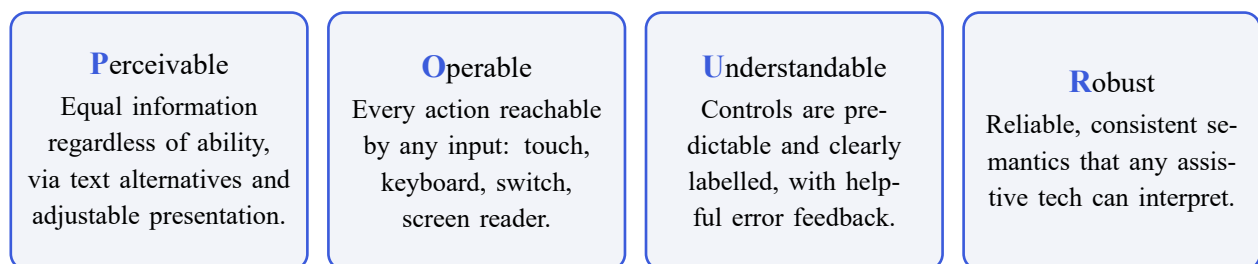


Figure 2: The four POUR principles that underpin WCAG.

2.3 Two trees, one screen

The decisive fact for an Android developer: what is drawn and what a screen reader traverses are *two separate trees* (Figure 3).

- **Composition tree** — the visual hierarchy; nodes are boxes, text, and images ordered by draw position.
- **Semantics tree** — runs in parallel; each node carries a name, role, state, and supported actions. It is the only thing assistive services can read.

TalkBack and Switch Access run as system services that subscribe to the *merged* semantics tree, turn each node into speech or a scan stop, and translate gestures back into actions. A pixel-perfect screen is unusable if its semantics tree is missing, mislabelled, or misordered. Every technique in this report shapes that tree.

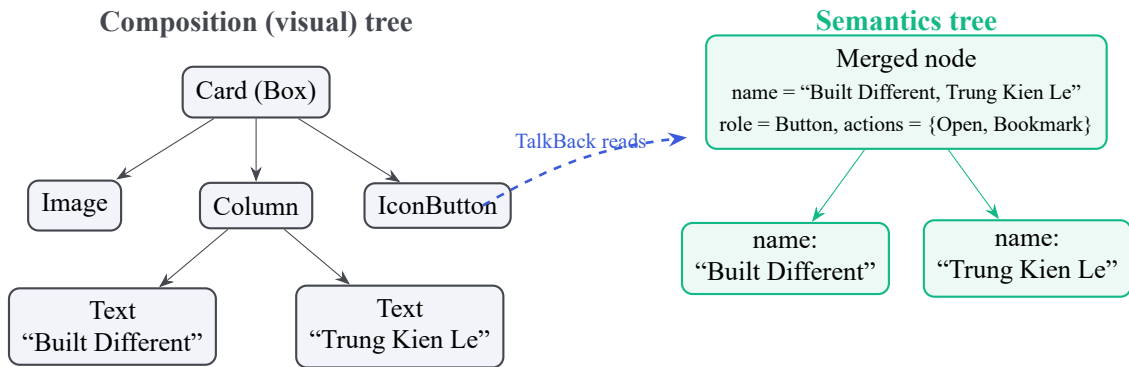


Figure 3: The visual tree (left) is drawn; the semantics tree (right) is read by TalkBack. The decorative image is dropped and the two texts are *merged* into one node — the techniques of §4 and §5.

2.4 Jetpack Compose

Compose is Android’s declarative UI toolkit: the UI is a function of state, and state changes trigger recomposition. It builds the semantics tree from Material components’ built-in semantics plus `Modifier.semantics` blocks — giving code-level control over exactly what assistive technology perceives.

3 The Case-Study App: *Accessible Podcast*

3.1 Overview

We built **Accessible Podcast**, a complete front-end Compose app, to ground the techniques. An in-memory mock implements the `PodcastRepository` interface, so the UI code is byte-for-byte identical to a real-backend version but runs anywhere with no configuration.

3.2 User flow

The app implements the classic three-step listening flow, plus a resume shortcut (Figure 4).



Figure 4: User flow. Each transition is reachable by TalkBack double-tap and by Switch Access.

3.3 Architecture

The code is organised into a thin data layer and a Compose UI layer (Figure 5). The UI depends only on the `PodcastRepository` interface, never on the mock, which is the “interface / mock” boundary that stands in for a backend.

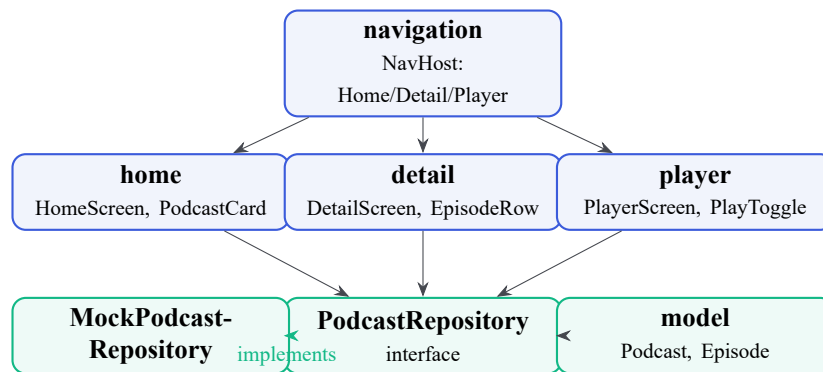


Figure 5: Package architecture. The Compose UI layer (blue) talks to a single repository interface; the mock (green) is the only implementation and can be swapped for a real one with no UI change.

3.4 The mock data layer

The repository exposes an observable catalogue as a `StateFlow`. Toggling a bookmark mutates the flow, so every screen recomposes exactly as it would against a reactive backend — which also lets us verify that state changes are announced to TalkBack.

```
interface PodcastRepository {
    val podcasts: StateFlow<List<Podcast>> // observable catalogue
    fun getEpisode(id: String): Episode?
    fun toggleBookmark(episodeId: String) // mutates state -> recomposition
    fun toggleDownload(episodeId: String)
}
```

Listing 1: The repository interface the whole UI depends on.

3.5 Where each technique lives

Table 1 maps every accessibility technique discussed in this report to the exact file in the project, so the demo and the prose stay in lock-step.

Report section	Technique	File
Core (§4)	<code>contentDescription</code> , <code>decorative</code> <code>null</code>	<code>ui/components/CoverArt.kt</code>
Core	<code>Role.Button</code> + <code>stateDescription</code>	<code>ui/player/PlayToggle.kt</code>
Core	48 dp touch targets	<code>ui/home/PodcastCard.kt</code>
Core	<code>sp</code> typography, font scaling	<code>ui/theme/Type.kt</code>
Core	heading navigation	<code>ui/home/HomeScreen.kt</code>
Advanced (§5)	<code>mergeDescendants</code>	<code>PodcastCard.kt</code> , <code>EpisodeRow.kt</code>
Advanced	<code>clearAndSetSemantics</code> + <code>customActions</code>	<code>EpisodeRow.kt</code>
Advanced	<code>hideFromAccessibility</code>	<code>CoverArt.kt</code>
Advanced	<code>isTraversalGroup</code> + <code>traversalIndex</code>	<code>HomeScreen.kt</code>
Background	<code>liveRegion</code> announcements	<code>ui/player/PlayerScreen.kt</code>
Testing (§6)	Compose assertions, ATF	<code>androidTest/AccessibilityTest.kt</code>

Table 1: Technique-to-file map. Every claim in the report is backed by runnable code.

4 Technical Approach I — Core Semantics

Five modifiers make a single component accessible — labelling, roles, state, touch-target size, and scalable type. They fix the most common and most damaging screen-reader defects.

4.1 Content descriptions: naming visual elements

A screen reader cannot see an icon; it can only read the icon’s text alternative — in Compose, the `contentDescription` parameter. Three rules make a good label:

- **Succinct & action-oriented.** Avoid “Image of” / “Button to”. The element’s name is announced separately.
- **Localized.** Always use `stringResource()` so labels translate with the device language.
- **Hide the decorative.** Purely decorative graphics must be removed from the tree so they do not add noise.

```
// Meaningful control – announced as "Search podcasts, button"
Icon(
    imageVector = Icons.Filled.Search,
    contentDescription = stringResource(R.string.cd_search)
)

// Decorative cover tile – next to a text title, so it adds nothing for TalkBack
Box(
    modifier = Modifier.size(56.dp).background(color)
    .semantics { hideFromAccessibility() } // Compose 1.8+; still visible to UI tests
)
```

Listing 2: Meaningful icons get a localized label; decorative art is hidden.

In practice: Instagram labels its action buttons (“Like”, “Comment”) but passes `null` to the decorative story gradients, so screen-reader users jump straight between real content.

4.2 Roles and state descriptions on custom controls

Material components carry their own semantics, but a custom control built from a bare `Box` tells assistive tech nothing. We must declare two things explicitly: a **role** (so TalkBack appends “Button”/“Switch”) and, for anything toggleable, a **state description** (the dynamic “Playing”/“Paused”). Our circular player button, taken directly from the app, does both:

```
Composable
fun PlayToggle(isPlaying: Boolean, onToggle: () -> Unit) {
    val name = stringResource(if (isPlaying) R.string.cd_pause else R.string.cd_play)
    val state = stringResource(if (isPlaying) R.string.state_playing else R.string.state_paused)
    Box(
        modifier = Modifier.size(72.dp).clip(CircleShape).background(primary)
        .clickable(onClick = onToggle, role = Role.Button)
        .semantics {
            contentDescription = name // "Play" / "Pause"
            stateDescription = state // "Playing" / "Paused"
            role = Role.Button
        },
        contentAlignment = Alignment.Center
    ) {
        Icon(imageVector = icon, contentDescription = null) // handled by the Box
    }
}
```



```
}
}
```

Listing 3: PlayToggle.kt — a custom control with an explicit role and a dynamic state.

TalkBack now announces “*Pause, button, Playing*” — name, role, and state — which is exactly how Spotify’s custom playback control behaves.

4.3 Touch-target sizing

Users with motor impairments, or anyone tapping on a moving bus, need a target they can hit reliably. Material Design mandates a minimum interactive size of **48 dp × 48 dp** (WCAG 2.5.5 asks for 44 px). The visual icon can stay small; only the *touchable area* must grow. Compose does this automatically for Material components and, for custom ones, via `minimumInteractiveComponentSize()`:

```
IconButton(
    onClick = onToggleBookmark,
    modifier = Modifier
        .minimumInteractiveComponentSize() // expands hit area to 48dp, no visual change
        .clearAndSetSemantics {}           // (see advanced section)
) { Icon(Icons.Filled.BookmarkBorder, contentDescription = null) }
```

Listing 4: A 24 dp icon with a guaranteed 48 dp touch target.

This is the same pattern Gmail uses for the small “star” icon: a 24 dp glyph with a 48 dp hit area, so users do not open the email by accident when they meant to star it.

4.4 Scalable typography

Android 14 introduced **non-linear font scaling** up to 200%: small text grows a lot while already-large headings grow less, preserving hierarchy. To benefit, code must separate *text size* from *layout size*:

1. Text sizes always in `sp` — Compose applies the non-linear math automatically.
2. Paddings and constraints always in `dp`.
3. **Never** hard-code a `height()` on a text container; use `defaultMinSize` or padding so it can grow.

Figure 6 shows why the third rule matters: a fixed-height card clips its text at 200%, while the `sp`-based card grows gracefully. Our `Type.kt` defines every style in `sp`, so the app passes this test out of the box.

4.5 Heading navigation

Finally, marking section titles with `semantics { heading() }` lets TalkBack users jump heading-to-heading instead of swiping through every item — the screen-reader equivalent of skimming. We apply it to the app-bar titles and the “Fresh for you” / “Episodes” headers.

5 Technical Approach II — Advanced Semantics & Navigation

Core semantics make one component correct; advanced semantics make a whole screen efficient to navigate. They control how many focus stops exist, the order they are read in, and how nested actions are reached — the difference between a technically-labelled UI and one that is fast to drive by swipe or switch.

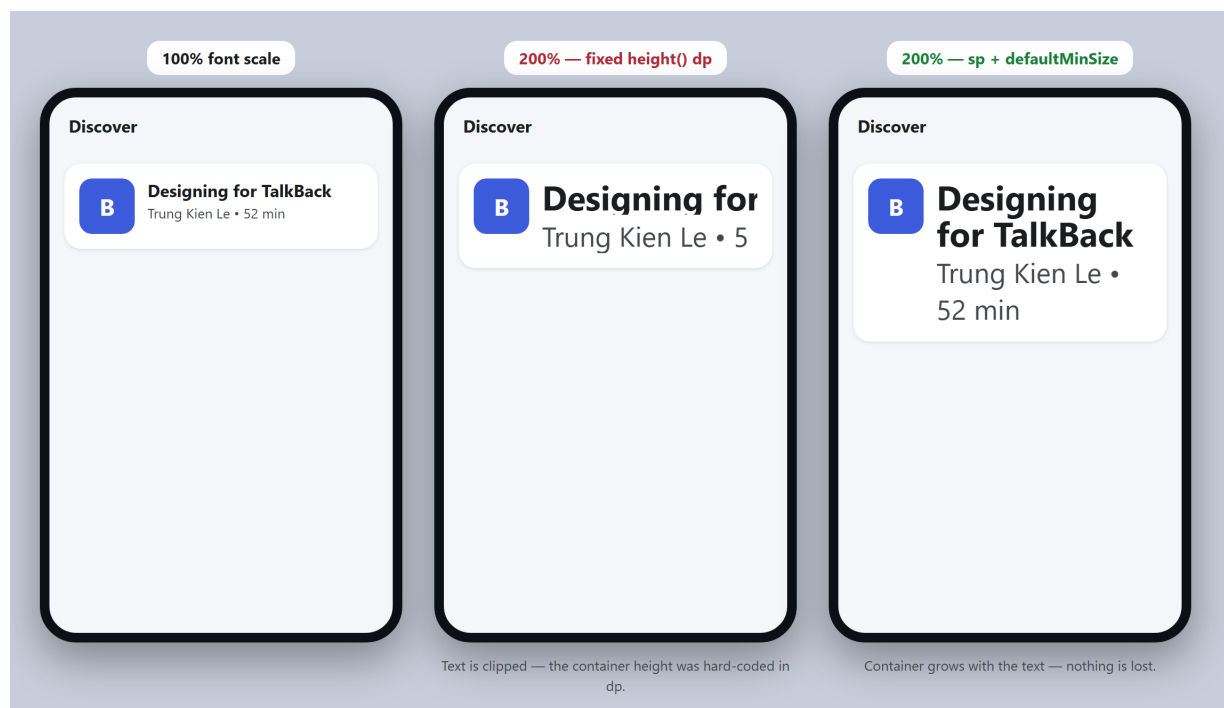


Figure 6: The same card at 100% and at 200% font scale. A hard-coded `height()` in dp (centre) clips the title; `sp` text with `defaultMinSize` (right) lets the container expand so nothing is lost.

5.1 Semantic merging: fewer, richer focus stops

By default every text node is its own focus stop. A podcast card with a cover, title, author, and metadata therefore costs a TalkBack user 4–6 swipes to cross — multiplied across a list of shows, this is exhausting. `Modifier.semantics(mergeDescendants = true)` collapses a container's children into one node whose announcement is the concatenation of its parts.

Before — no merging

Every text and icon is its own focus stop. TalkBack needs 6 swipes to cross one card.

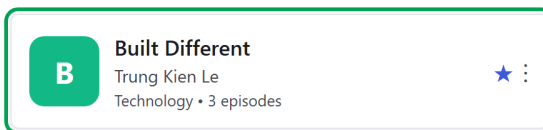


6 focus stops • 6 swipes per card

TalkBack says: "Built Different" (swipe) "Trung Kien Le" (swipe) "Technology, 3 episodes" (swipe) "Star, button" ...

After — mergeDescendants = true

The card is one focus stop; nested actions move to the TalkBack actions menu.



1 focus stop • 1 swipe per card

TalkBack says: "Built Different, Trung Kien Le, Technology, 3 episodes." "Actions available" → Bookmark, More options.

Figure 7: The effect of `mergeDescendants = true` on a podcast card. Before (left): six separate focus stops, six swipes. After (right): one focus stop that TalkBack reads as a single sentence, with the nested bookmark/menu actions moved into the actions menu.

```
Card(
    onClick = onOpen,
    modifier = Modifier.semantics(mergeDescendants = true) {
        customActions = listOf(
            CustomAccessibilityAction(bookmarkLabel) { onToggleBookmark(latest); true }
        )
    }
)
```

```

    )
  }
) { /* cover + title + author + metadata + nested icon buttons */ }

```

Listing 5: PodcastCard.kt — the card is one focus stop.

Constraint. A parent cannot merge a child that already merges itself. Interactive components (`Button`, `Switch`, anything with `clickable` / `toggleable` modifiers) resist merging so they remain independently operable. This is why the nested bookmark button needs the next technique.

5.2 Clearing and hiding: `clearAndSetSemantics` & `hideFromAccessibility`

Two modifiers prune the tree:

- `clearAndSetSemantics {}` wipes a node’s semantics (and its descendants’) and lets us redefine them or leave them empty to remove the node from traversal entirely.
- `hideFromAccessibility()` (Compose 1.8+) hides a node from assistive services while keeping it for UI tests. This is the modern alternative to setting `contentDescription` to `null` on decorative elements.

In `CoverArt.kt` the coloured cover tile is decorative when a text title sits beside it, so we hide it. In `EpisodeRow.kt` the three trailing icon buttons are cleared so they do not each steal focus — their actions are re-exposed on the row (next subsection).

5.3 Custom accessibility actions

Nested icon buttons are an anti-pattern for screen readers: they are tiny targets and they clutter the swipe path (50 cards × 3 buttons = 150 extra swipes). The fix is to *clear* the physical buttons and *elevate* their operations to `customActions` on the parent row, where `TalkBack` surfaces them through its actions menu (“Actions available”).

```

Row(
    modifier = Modifier.semantics(mergeDescendants = true) {
        customActions = listOf(
            CustomAccessibilityAction(playLabel) { onPlay(); true },
            CustomAccessibilityAction(bookmarkLabel) { onToggleBookmark(); true },
            CustomAccessibilityAction(downloadLabel) { onToggleDownload(); true },
        )
    }
) {
    FilledIconButton(onClick = onPlay,
        modifier = Modifier.size(48.dp).clearAndSetSemantics {}) { /* icon */ }
    // ... title + metadata ...
    IconButton(onClick = onToggleBookmark,
        modifier = Modifier.minimumInteractiveComponentSize().clearAndSetSemantics {}) { /* icon */ }
}

```

Listing 6: EpisodeRow.kt — nested actions promoted to the row’s actions menu.

A screen-reader user now hears the episode as one sentence, then opens “Actions” to Play, Bookmark, or Download — while sighted users still tap the visible 48 dp buttons directly.

5.4 Traversal order

Assistive tech reads geometrically (top-to-bottom, left-to-right), which breaks on non-linear layouts. Two modifiers fix it: `isTraversalGroup = true` draws a boundary that must be read completely before mov-

ing on, and `traversalIndex` (a `Float`, lowest first) reorders within that boundary. A common pitfall is setting `traversalIndex` *without* a group: because all nodes default to `0f` globally, the element is compared against the whole screen and jumps to the bottom. On the Discover screen we make the “Continue listening” banner read *first* even though it is not the top-most focusable item:

```
LazyColumn(modifier = Modifier.semantics { isTraversalGroup = true }) {
    item {
        ContinuelisteningBanner(
            modifier = Modifier.semantics { traversalIndex = -1f } // read before everything else
        )
    }
    // ... section header + podcast cards ...
}
```

Listing 7: `HomeScreen.kt` — resume banner announced first via a low traversal index.

5.5 Decision matrix

Table 2 summarises when to reach for each advanced modifier.

API	Applied to	Use case / effect
<code>mergeDescendants = true</code> <code>clearAndSetSemantics {}</code>	Container (Row/Column/Card) Custom or nested nodes	Fuse fragmented text into one focus stop. Erase raw semantics; redefine or remove from traversal.
<code>hideFromAccessibility()</code> <code>isTraversalGroup = true</code>	Decorative elements Layout container	Hide from TalkBack, keep for UI tests. Force children to be read as a bounded block.
<code>traversalIndex = Xf</code> <code>customActions = listOf(...)</code>	Child of a traversal group Parent list item / card	Reorder reading (lowest value read first). Move nested actions into the TalkBack menu.

Table 2: Choosing an advanced semantics API by structural need.

6 Testing & Evaluation

No single tool certifies accessibility: automated checkers catch only a fraction of real barriers (W3C’s own position), so the rest needs human judgement and task evidence. We therefore **triangulate** across four layers (Figure 8) and — just as important — separate *raw suggestions* from *confirmed defects*.

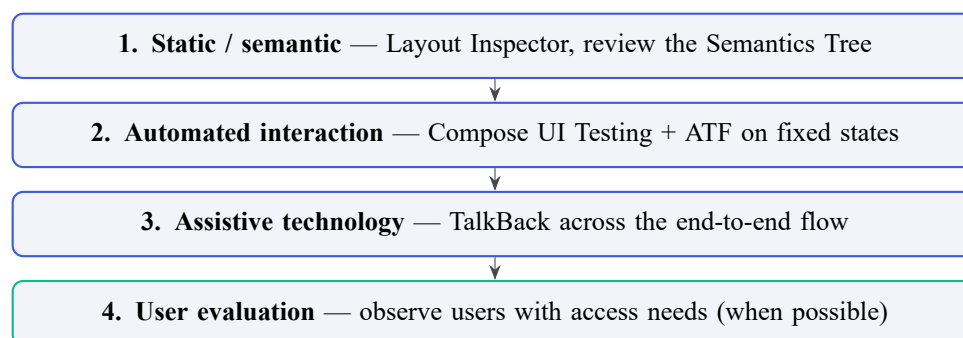


Figure 8: The layered evaluation model. Each layer answers a different question; no single layer is sufficient on its own.

6.1 Manual testing with TalkBack

TalkBack is the ground truth for the listening experience. Our protocol runs each core flow twice: first swiping linearly to check focus order and labels, then exploring by touch to compare the visual region with the focus region. For every important control we activate it and verify the role, state, and *post-action focus*. Common issues this surfaces — and which pure automation misses — are wrong or missing labels, focus that jumps after a dialog, over-merged cards that hide a child action, and dynamic changes (buffering, errors) that are never announced. We record each finding with a severity, reproduction steps, and expected-vs-actual behaviour.

6.2 Accessibility Scanner — and an honest case study

Accessibility Scanner evaluates a screen on-device and draws orange boxes around potential issues (labels, touch targets, contrast). To practise the *evidence-collection workflow* — not to judge another company’s app — we captured a 15-screen flow through Agoda (Figure 9). The point of the exercise is methodological: how to record, de-duplicate, and interpret tool output responsibly.

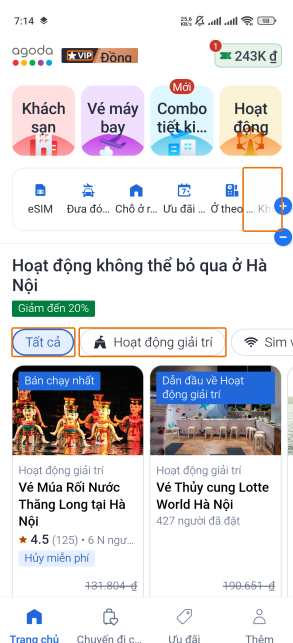


Figure 9: Accessibility Scanner on a real app (Agoda), used only to rehearse the workflow. Orange boxes mark raw *suggestions* — e.g. a 38 dp touch target below the 48 dp guideline and chips that may lack a distinct label. Raw suggestions are not confirmed defects.

The 15-screen capture produced **102 raw suggestions**, distributed as in Figure 10. These numbers must be read carefully: they are *not* 102 independent, confirmed bugs. Loading/skeleton states, web-view content, and the *same* chip appearing across several frames all inflate the count (report 2 and report 14 overlap; the splash screen produced none). The correct next step is de-duplication by root cause and confirmation with TalkBack — never turning a warning into a conclusion without reproducing it.

6.3 Layout Inspector, Compose UI Testing, and ATF

Once we have our own source, three developer tools close the loop:

- **Layout Inspector** shows the live merged and unmerged semantics trees, linking a TalkBack/Scanner symptom to a specific node — a diagnostic tool, not a verdict on user experience.

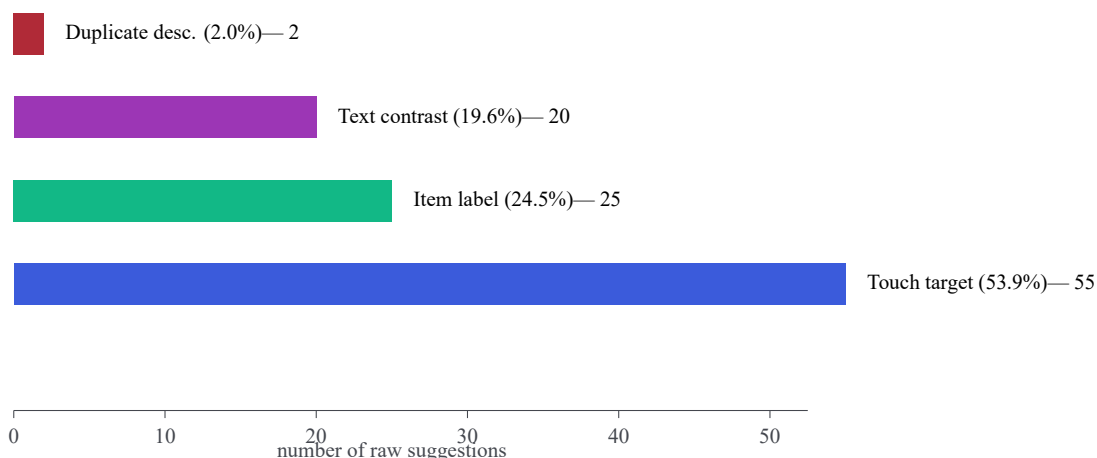


Figure 10: Distribution of the 102 raw suggestions across the 15-screen Agoda capture. Touch-target warnings dominate, but this is a *starting hypothesis*, not a root-cause finding.

- **Compose UI Testing** asserts semantics contracts in CI: that a control has the accessible name a screen reader will speak, and `printToLog()` dumps the tree for diagnosis. Our `AccessibilityTest.kt` does exactly this.
- **Accessibility Test Framework (ATF)**, Google’s open-source rule engine, adds automated checks for speakable text, touch-target size, and contrast, usable as a CI quality gate.

```
// 1. The icon-only search button must expose a spoken label, not just a glyph.
composeRule.onNodeWithContentDescription("Search podcasts").assertExists()

// 2. Dump the unmerged tree to verify merge/clear behaviour on the cards.
composeRule.onRoot(useUnmergedTree = true).printToLog("A11Y_TREE")
```

Listing 8: `AccessibilityTest.kt` — assert a spoken name, then dump the tree.

Table 3 positions each tool against the others.

Tool	Source?	Mode	Best for / cannot replace
TalkBack	No	Manual	Listening experience, focus, end-to-end flow.
Accessibility Scanner	No	Semi-auto	Fast visual triage of labels/targets/contrast; not coverage.
Layout Inspector	Debug build	Manual	Inspect merged/unmerged tree; not user experience.
Compose UI Testing	Yes	Automated	Semantics contracts & regression; not utterance quality.
ATF	Yes	Automated	Standardised checks in CI; not real usability.

Table 3: Complementary roles of the five verification tools.

6.4 Evaluation metrics and trade-offs

A credible evaluation states scope, representative flows, device, versions, and completion criteria, then reports both automated and task-based measures: first-flow coverage, task-completion rate, completion time, number of TalkBack gestures, automated pass rate, and a *confirmed-defect rate* that distinguishes real bugs from raw suggestions. Findings are triaged by **impact on the task** — *Blocker* (task impossible, fix before ship) > *Major* > *Moderate* > *Minor* — not merely by count. A missing label on the checkout button outweighs many touch-target warnings on secondary content.

Every technique in this report is a trade-off, and naming them is part of an honest evaluation:

- `mergeDescendants` cuts swipes but can hide an independent child action or produce an over-long sentence — measure gesture count *and* listen to the whole utterance.
- Rich labels aid comprehension but, if they duplicate role/state, add listening time.
- `traversalIndex` fixes reading order but hand-tuned indices are brittle and can drift from the visual layout; use the minimum number and document why.
- Custom controls give design freedom but must re-declare role, state, and keyboard behaviour that Material would have provided for free.

7 Conclusion

Accessibility in Compose is not a checkbox but control over the semantics tree at every level: naming and hiding nodes correctly (§4), then grouping, ordering, and consolidating their actions (§5). The payoff is measurable — a podcast card drops from six focus stops to one, the resume banner is announced first, and playback state is spoken the instant it changes — so a TalkBack user completes the full Discover → Detail → Player flow without sight.

Verification matters as much as construction (§6): TalkBack judges the listening experience, Scanner and ATF triage mechanical defects, Layout Inspector explains structure, and Compose tests lock semantics against regressions. No layer suffices alone, and — as the Agoda capture showed — a tool’s raw suggestion is a hypothesis, not a defect.

The practical lesson: build accessibility in from the first composable. Prefer Material components for their default semantics, add custom semantics only where the design forces it, and test with the assistive technology users actually run.

Future work

Version 2: run TalkBack on our own build across a device/version matrix, capture Layout Inspector evidence for representative findings, wire ATF into CI, and — resources permitting — have users who rely on assistive technology evaluate the core tasks.

Appendix on AI usage

AI assistants were used as a drafting and formatting aid: structuring the report, generating the LaTeX scaffolding and diagrams, and rendering the annotated UI mockups from our own designs and code. The accessibility techniques, app architecture, research decisions, and case-study data are the group’s own work; every AI-assisted passage was reviewed and edited by the members responsible for that section. Code samples are excerpts from the app we wrote, not AI-generated boilerplate.

8 Appendix A — Running the app

1. Open the `PodcastAccessibilityApp` folder in Android Studio (Koala or newer); let it sync and generate the Gradle wrapper and `local.properties`.
2. Run the app configuration on an emulator or device (minSdk 24, targetSdk 34).
3. To experience the accessibility work, enable **TalkBack**, install **Accessibility Scanner**, and set the system **font size** to 200%.

4. Run the instrumented checks: `./gradlew connectedAndroidTest`.

Project layout. `data/` holds the models and the `PodcastRepository` interface with its mock; `ui/` holds the theme, navigation, and the home, detail, and player screens. Section 3, Table 1 maps every technique to its file.

Limitations of this report

This is a front-end study. The app ships with mocked data and no real audio engine, so performance and networked-state accessibility (e.g. announcing buffering under poor connectivity) are described at the design level rather than measured. The Agoda case study used a single device and one capture pass; its figures demonstrate the evidence-collection *process* and must not be read as an audit of Agoda. Absolute WCAG conformance is likewise not claimed — only a principled mapping of technology-neutral criteria onto Android.

References

- [1] Android Developers. *Principles for improving app accessibility*. developer.android.com. Accessed 22 Jul 2026.
- [2] Android Developers. *Semantics in Compose*. developer.android.com/develop/ui/compose/semantics.
- [3] Google. *Get started on Android with TalkBack*. Android Accessibility Help.
- [4] Android Developers. *Merging and clearing (Jetpack Compose accessibility)*. developer.android.com.
- [5] Google. *Accessibility Scanner*. Google Play / Android Accessibility Help.
- [6] Android Developers. *Accessibility Scanner — Accessibility on Android* (video).
- [7] Android Developers. *Debug your layout with Layout Inspector* (Compose semantics view).
- [8] Android Developers. *Accessibility in Compose — key concepts and the semantics tree*.
- [9] Android Developers. *Test your Compose layout — merged and unmerged semantics trees*.
- [10] Android Developers. *Modify traversal order* (`isTraversalGroup` , `traversalIndex`).
- [11] Android Developers. *Testing your Compose layout — semantics matchers, assertions, printToLog*.
- [12] Android Developers. *Automated accessibility checks in Compose tests* (ATF integration, Compose 1.8+).
- [13] Google. *Accessibility Test Framework for Android* (open-source repository).
- [14] Android Developers. *Accessibility checking with Espresso* (`AccessibilityChecks.enable()`).
- [15] W3C Web Accessibility Initiative (WAI). *Selecting Web Accessibility Evaluation Tools* — automation covers only part of conformance.
- [16] W3C. *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation, 2023.
- [17] W3C. *WCAG2ICT — Applying WCAG to Non-Web Information and Communications Technologies*.
- [18] W3C WAI. *Website Accessibility Conformance Evaluation Methodology (WCAG-EM) 1.0*.
- [19] W3C WAI. *Involving Users in Evaluating Web Accessibility*.
- [20] ISO. *ISO 9241-11:2018 — Usability: definitions and concepts* (effectiveness, efficiency, satisfaction).
- [21] W3C WAI. *Mobile Accessibility: How WCAG 2 and other WAI guidelines apply to mobile*.
- [22] Google. *Material Design — Accessibility* (touch targets, contrast).
- [23] European Commission. *European Accessibility Act (Directive (EU) 2019/882)*.
- [24] Android Developers. *Android 14 features — non-linear font scaling*.
- [25] Appt.org. *Accessibility action in Jetpack Compose*.